C Programming

Kriti Ghimire

Westcliff University

Fundamentals of C Programming

Nirajan Thakuri

April 04, 2026

Q1. Write a C program to find the maximum and minimum elements in an array.

```c
#include <stdio.h>
int main() {
    int n, i;

    // Input size of array
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    // Input array elements
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int max = arr[0];
    int min = arr[0];

    for(i = 1; i < n; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
        if(arr[i] < min) {
            min = arr[i];
        }
    }
    printf("Maximum element = %d\n", max);
    printf("Minimum element = %d\n", min);

    return 0;
}
```

Output

```
 } ; if ($?) { .\q1 }
Enter number of elements: 5
Enter 5 elements:
45 66 88 61 21
Maximum element = 88
Minimum element = 21
PS C:\Users\kriti\OneDrive\Desktop\
```

Q2. Implement a C program to reverse the elements of an array.

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    for(i = 0; i < n / 2; i++) {
        int temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }
    printf("Reversed array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Ouput

```
 .\q2 }
Enter number of elements: 5
Enter 5 elements:
6 5 4 3 2
Reversed array:
2 3 4 5 6
PS C:\Users\kriti\OneDrive\Deskt
```

Q3. Write a C program to sort an array of integers in ascending order using a sorting algorithm of your choice (e.g., bubble sort, selection sort, insertion sort).

```c
#include <stdio.h>

int main() {
    int n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Bubble sort logic
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    printf("Sorted array in ascending order:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

```
Enter number of elements: 4
Enter 4 elements:
3 1 9 4
Sorted array in ascending order:
1 3 4 9
PS C:\Users\kriti\OneDrive\Deskto
nt-3-Kritigh62-bit\src>
```

Q4. Implement a C program to find the second largest element in an array.

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Initialize largest and second largest
    int largest = arr[0];
    int secondLargest = -2147483648; // minimum int value

    for(i = 1; i < n; i++) {
        if(arr[i] > largest) {
            secondLargest = largest;
            largest = arr[i];
        }
        else if(arr[i] > secondLargest && arr[i] != largest) {
            secondLargest = arr[i];
        }
    }

    if(n < 2) {
        printf("Array must have at least two elements.\n");
    } else {
        printf("Second largest element = %d\n", secondLargest);
    }

    return 0;
}
```

Output

```
.\q4 }
Enter number of elements: 4
Enter 4 elements:
23 11 44 54
Second largest element = 44
PS C:\Users\kriti\OneDrive\Deskto
nt-3-Kritigh62-bit\src>
```

Q5. Write a C program to merge two sorted arrays into a single sorted array.

```c
#include <stdio.h>

int main() {
    int n1, n2;

    printf("Enter size of first and second array: ");
    scanf("%d %d", &n1, &n2);

    int arr1[n1], arr2[n2];

    printf("Enter value for first array: ");
    for(int i=0; i<n1; i++) scanf("%d", &arr1[i]);

    printf("Enter value for second array: ");
    for(int j=0; j<n2; j++) scanf("%d", &arr2[j]);

    int merged_arr[n1+n2];

    int x=0, y=0, z=0;

    while (x<n1 || y<n2) {
        if (arr1[x] < arr2[y]) merged_arr[z++] = arr1[x++];
        else merged_arr[z++] = arr2[y++];
    }

    while (x<n1) merged_arr[z++] = arr1[x++];
    while (y<n2) merged_arr[z++] = arr2[y++];

    printf("Merged array: ");
    for(int k=0; k<n1+n2; k++) printf("%d ", merged_arr[k]);
    printf("\n");

    return 0;
}
```

Output

```
 .\q5 }
Enter size of first and second array: 3 1
Enter value for first array: 1 2 3
Enter value for second array: 4
Merged array: 1 2 3 4
PS C:\Users\kriti\OneDrive\Desktop\C\assignm
```

Q6. Write a C program to find the length of a string without using the built-in string functions.

```c
#include <stdio.h>

int main() {
    char str[100];
    int length = 0;

    // Input string
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Calculate length manually
    while (str[length] != '\0') {
        length++;
    }

    printf("Length of the string = %d\n", length);

    return 0;
}
```

Output

```
 .\q6 }
Enter a string: programming
Length of the string = 12
PS C:\Users\kriti\OneDrive\Desktop\C\assig
nt-3-Kritigh62-bit\src>
```

Q7. Implement a C program to reverse a string.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, length;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);


    str[strcspn(str, "\n")] = '\0';

    length = strlen(str);

    // Reverse the string
    for(i = 0; i < length / 2; i++) {
        char temp = str[i];
        str[i] = str[length - i - 1];
        str[length - i - 1] = temp;
    }

    printf("Reversed string: %s\n", str);

    return 0;
}
```

Output

```
rc\" ; if ($?) { gcc q7.c -o q7 } ; if
 .\q7 }
Enter a string: apple
Reversed string: elppa
PS C:\Users\kriti\OneDrive\Desktop\C\a
nt-3-Kritigh62-bit\src>
```

Q8. Write a C program to check if a given string is a palindrome.

```c
#include <stdio.h>

int main() {
    char str[100];
    int i = 0, length = 0, isPalindrome = 1;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    while (str[length] != '\0') {
        length++;
    }

    // Check palindrome
    for (i = 0; i < length / 2; i++) {
        if (str[i] != str[length - i - 1]) {
            isPalindrome = 0;
            break;
        }
    }

    if (isPalindrome)
        printf("The string is a palindrome.\n");
    else
        printf("The string is not a palindrome.\n");

    return 0;
}
```

Output

```
Enter a string: eye
The string is a palindrome.
PS C:\Users\kriti\OneDrive\Deskto
nt-3-Kritigh62-bit\src>
```

```
nnerFile }
Enter a string: cat
The string is not a palindrome.
PS C:\Users\kriti\OneDrive\Desktop\C\ass
nt-3-Kritigh62-bit\src>
```

Q9. Implement a C program to count the occurrence of a specific character in a string.

```c
#include <stdio.h>

int main() {
    char str[100], ch;
    int i = 0, count = 0;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    printf("Enter the character to count: ");
    scanf(" %c", &ch);

    // Count occurrences
    while (str[i] != '\0') {
        if (str[i] == ch) {
            count++;
        }
        i++;
    }

    printf("Character '%c' occurs %d times.\n", ch, count);

    return 0;
}
```

Output

```
 .\q9 }
Enter a string: programming
Enter the character to count: g
Character 'g' occurs 2 times.
PS C:\Users\kriti\OneDrive\Desktop\C\a
nt-3-Kritigh62-bit\src>
```

Q10. Write a C program to concatenate two strings without using the built-in string functions.

```c
#include <stdio.h>
int main() {
    char str1[100], str2[100];
    int i = 0, j = 0;

    printf("Enter first string: ");
    scanf("%[^\n]", str1);

    getchar();

    printf("Enter second string: ");
    scanf("%[^\n]", str2);

    while (str1[i] != '\0') {
        i++;
    }


    while (str2[j] != '\0') {
        str1[i] = str2[j];
        i++;
        j++;
    }

    str1[i] = '\0';

    printf("Concatenated string: %s\n", str1);

    return 0;
}
```

Output

```
rc\" ; if ($?) { gcc q10.c -o q10 } ; if
 { .\q10 }
Enter first string: New
Enter second string:  Year
Concatenated string: New  Year
PS C:\Users\kriti\OneDrive\Desktop\C\ass
nt-3-Kritigh62-bit\src>
```

Q11. Write a function named calculateAverage that takes an array of integers as input and returns the average of the numbers.

```c
#include <stdio.h>
float calculateAverage(int arr[], int size) {
    int i;
    int sum = 0;

    for (i = 0; i < size; i++) {
        sum += arr[i];
    }

    return (float)sum / size;
}

int main() {
    int arr[100], n, i;
    float avg;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    avg = calculateAverage(arr, n);

    printf("Average = %.2f\n", avg);

    return 0;
}
```

Output

```
Enter number of elements: 6
Enter 6 integers:
4 6 3 7 5 9
Average = 5.67
PS C:\Users\kriti\OneDrive\Desktop\
nt-3-Kritigh62-bit\src>
```

Q12. Write a function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```c
#include <stdio.h>
int isPalindrome(char str[]) {
    int i = 0, length = 0;

    while (str[length] != '\0') {
        length++;
    }

    for (i = 0; i < length / 2; i++) {
        if (str[i] != str[length - i - 1]) {
            return 0;
        }
    }

    return 1;
}

int main() {
    char str[100];

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    printf("%d\n", isPalindrome(str));

    return 0;
}
```

Output

```
rc\" ; if ($?) { gcc q12.c -o
 { .\q12 }
Enter a string: data
0
PS C:\Users\kriti\OneDrive\De
nt-3-Kritigh62-bit\src>
```

Q13. Write a function named findFactorial that takes an integer as input and returns its factorial.

```c
#include <stdio.h>

int findFactorial(int n);

int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Factorial = %d\n", findFactorial(n));

    return 0;
}

int findFactorial(int n) {
    int i;
    int fact = 1;

    for (i = 1; i <= n; i++) {
        fact = fact * i;
    }

    return fact;
}
```

Output

```
tempCodeRunnerFile } ; if ($?)
nnerFile }
Enter a number: 6
Factorial = 720
PS C:\Users\kriti\OneDrive\Des
nt-3-Kritigh62-bit\src>
```

Q14. Write a function named convertTemperature that takes a temperature value in Celsius and converts it to Fahrenheit. The function should return the converted temperature.

```c
#include <stdio.h>

float convertTemperature(float celsius);

int main() {
    float celsius;

    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);

    printf("Temperature in Fahrenheit = %.2f\n", convertTemperature(celsius));

    return 0;
}
float convertTemperature(float celsius) {
    float fahrenheit;

    fahrenheit = (celsius * 9 / 5) + 32;

    return fahrenheit;
}
```

Output

```
 { .\q14 }
Enter temperature in Celsius: 39
Temperature in Fahrenheit = 102.20
PS C:\Users\kriti\OneDrive\Desktop\C\as
```

Q15. Write a function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```c
#include <stdio.h>

int countOccurrences(char str[], char ch);

int main() {
    char str[100], ch;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    printf("Enter a character to count: ");
    scanf(" %c", &ch);

    printf("Character '%c' occurs %d times.\n", ch, countOccurrences(str, ch));

    return 0;
}

// Function definition
int countOccurrences(char str[], char ch) {
    int i = 0, count = 0;

    while (str[i] != '\0') {
        if (str[i] == ch) {
            count++;
        }
        i++;
    }

    return count;
}
```

Output

```
 { .\q15 }
Enter a string: rhododendron
Enter a character to count: d
Character 'd' occurs 3 times.
PS C:\Users\kriti\OneDrive\Deskt
```

Q16. Write a function named reverseArray that takes an array of integers as input and reverses the order of the elements in the array.

```c
#include <stdio.h>

void reverseArray(int arr[], int size);

int main() {
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    reverseArray(arr, n);

    printf("Reversed array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}

void reverseArray(int arr[], int size) {
    int i, temp;

    for (i = 0; i < size / 2; i++) {
        temp = arr[i];
        arr[i] = arr[size - i - 1];
        arr[size - i - 1] = temp;
    }
}
```

Output

```
Enter number of elements: 6
Enter 6 integers:
3 4 5 6 1 2
Reversed array: 2 1 6 5 4 3
PS C:\Users\kriti\OneDrive\Desktop\C\assignme
nt-3-Kritigh62-bit\src>
```

Q17. Write a function named calculatePower that takes two integers, base and exponent, as input and returns the result of raising the base to the exponent.

```c
#include <stdio.h>

int calculatePower(int base, int exponent) {
    int result = 1;
    for (int i=0;i<exponent; i++) {
        result *= base;
    }
    return result;
}

int main() {
    int b, e;
    printf("Enter base and exponent: ");
    scanf("%d %d", &b, &e);

    int power = calculatePower(b, e);
    printf("Result is: %d.\n", power);

    return 0;
}
```

Output

```
 { .\q17 }
Enter base and exponent: 2 4
Result is: 16.
PS C:\Users\kriti\OneDrive\Deskto
nt-3-Kritigh62-bit\src>
```

Q18. Write a function named findPrimeNumbers that takes an integer n as input and prints all prime numbers from 1 to n.

```c
#include <stdio.h>
int findPrimeNumbers(int n) {
    int i, j, isPrime;

    for (i = 2; i <= n; i++) {
        isPrime = 1;

        for (j = 2; j <= i / 2; j++) {
            if (i % j == 0) {
                isPrime = 0;
                break;
            }
        }
          if (isPrime == 1) {
            printf("%d ", i);
        }
    }

    return 0;

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Prime numbers from 1 to %d are: ", n);

    findPrimeNumbers(n);

    printf("\n");

    return 0;
}
```

Output

```
rc\" ; if ($?) { gcc q18.c -o q18 } ; if ($?)
 { .\q18 }
Enter a number: 20
Prime numbers from 1 to 20 are: 2 3 5 7 11 13
 17 19
PS C:\Users\kriti\OneDrive\Desktop\C\assignme
nt-3-Kritigh62-bit\src>
```

Q19. Write a function named calculateFactorialSeries that takes an integer n as input and prints the factorial series up to n.

```c
#include <stdio.h>

int calculateFactorialSeries(int n) {
    int i, j, fact;

    printf("Factorial series: ");

    for (i = 1; i <= n; i++) {
        fact = 1;

        for (j = 1; j <= i; j++) {
            fact = fact * j;
        }

        printf("%d ", fact);
    }

    return 0;
}

int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    calculateFactorialSeries(n);

    printf("\n");

    return 0;
}
```

Output

```
 { .\q19 }
Enter a number: 7
Factorial series: 1 2 6 24 120 720 5040
PS C:\Users\kriti\OneDrive\Desktop\C\assig
nt-3-Kritigh62-bit\src>
```

Q20. Write a function named calculateGCD that takes two integers as input and returns their greatest common divisor (GCD).

```c
#include <stdio.h>
int calculateGCD(int a, int b);

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("GCD of %d and %d is %d\n", a, b, calculateGCD(a, b));

    return 0;
}


int calculateGCD(int a, int b) {
    int i, gcd = 1;

    for (i = 1; i <= a && i <= b; i++) {
        if (a % i == 0 && b % i == 0) {
            gcd = i;
        }
    }

    return gcd;
}
```

Output

```
Enter two numbers: 4 6
GCD of 4 and 6 is 2
PS C:\Users\kriti\OneDrive\Desk
nt-3-Kritigh62-bit\src>
```

Q21. Write a recursive function named calculateFactorial that takes an integer n as input and returns its factorial.

```c
#include <stdio.h>

int calculateFactorial(int n);

int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Factorial of %d is %d\n", n, calculateFactorial(n));

    return 0;
}

// Recursive function
int calculateFactorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * calculateFactorial(n - 1);
}
```

Output

```
Enter a number: 4
Factorial of 4 is 24
PS C:\Users\kriti\OneDrive\Desk
nt-3-Kritigh62-bit\src>
```

Q22. Write a recursive function named calculateFibonacci that takes an integer n as input and returns the nth Fibonacci number. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the two preceding numbers.

```c
#include <stdio.h>

int calculateFibonacci(int n);

int main() {
    int n, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci series: ");

    for (i = 0; i < n; i++) {
        printf("%d ", calculateFibonacci(i));
    }

    printf("\n");

    return 0;
}

// Recursive function
int calculateFibonacci(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return calculateFibonacci(n - 1) + calculateFibonacci(n - 2);
}
```

Output

```
 { .\q22 }
Enter number of terms: 7
Fibonacci series: 0 1 1 2 3 5 8
PS C:\Users\kriti\OneDrive\Desktop\C\assignme
nt-3-Kritigh62-bit\src>
```

Q23. Write a recursive function named calculateGCD that takes two integers a and b as input and returns their greatest common divisor (GCD).

```c
#include <stdio.h>

int calculateGCD(int a, int b);

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("GCD of %d and %d is %d\n", a, b, calculateGCD(a, b));

    return 0;
}

int calculateGCD(int a, int b) {
    if (b == 0)
        return a;
    else
        return calculateGCD(b, a % b);
}
```

Output

```
Enter two numbers: 6 9
GCD of 6 and 9 is 3
PS C:\Users\kriti\OneDrive\Desktop\C\a
nt-3-Kritigh62-bit\src>
main*  ⊗ 0 ⚠ 0        Spaces: 4    UTF-8
```

Q24. Write a recursive function named calculatePower that takes two integers base and exponent as input and returns the result of raising the base to the exponent.

```c
#include <stdio.h>

int calculatePower(int base, int exponent) {
    if (exponent == 0)
        return 1;
    return base * calculatePower(base, exponent - 1);
}

int main() {
    int base, exponent;

    printf("Enter base: ");
    scanf("%d", &base);

    printf("Enter exponent: ");
    scanf("%d", &exponent);

    if (exponent < 0) {
        printf("Negative exponents not supported.\n");
    } else {
        int result = calculatePower(base, exponent);
        printf("Result: %d\n", result);
    }

    return 0;
}
```

Output

```
nter base: 5
nter exponent: 7
esult: 78125
S C:\Users\kriti\OneDrive\Desktop\
t-3-Kritigh62-bit\src>
```

Q25. Write a recursive function named calculateSumOfDigits that takes an integer n as input and returns the sum of its digits.

```c
#include <stdio.h>

int calculateSumOfDigits(int n);

int main() {
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Sum of digits = %d\n", calculateSumOfDigits(n));

    return 0;
}

int calculateSumOfDigits(int n) {
    if (n == 0)
        return 0;
    else
        return (n % 10) + calculateSumOfDigits(n / 10);
}
```

Output

```
 { .\q25 }
Enter a number: 58
Sum of digits = 13
PS C:\Users\kriti\OneDrive\Des
```

Q26. Write a recursive function named reverseString that takes a string as input and returns the reversed string.

```c
#include <stdio.h>

void reverseString(char str[], int start, int end);

int main() {
    char str[100];
    int length = 0;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    while (str[length] != '\0') {
        length++;
    }

    reverseString(str, 0, length - 1);

    printf("Reversed string: %s\n", str);

    return 0;
}

void reverseString(char str[], int start, int end) {
    char temp;

    if (start >= end)
        return;

    temp = str[start];
    str[start] = str[end];
    str[end] = temp;

    reverseString(str, start + 1, end - 1);
}
```

Output

```
 { .\q26 }
Enter a string: intel
Reversed string: letni
PS C:\Users\kriti\OneDrive\Deskto
nt-3-Kritigh62-bit\src>
```

Q27. Write a recursive function named printTriangle that takes an integer n as input and prints a triangle of asterisks (*) with n rows.

```c
#include <stdio.h>
int printTriangle(int n);

int main() {
    int n;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    printTriangle(n);

    return 0;
}

// Recursive function
int printTriangle(int n) {
    if (n == 0)
        return 0;

    printTriangle(n - 1);

    for (int i = 1; i <= n; i++) {
        printf("* ");
    }

    printf("\n");

    return 0;
}
```

Output

```
Enter number of rows: 5
*
* *
* * *
* * * *
* * * * *
PS C:\Users\kriti\OneDrive\Desktop\C\assign
```

Q28. Write a recursive function named calculateBinary that takes an integer n as input and returns its binary representation as a string.

```c
#include <stdio.h>
#include <string.h>

void calculateBinary(int n, char *result) {
    if (n == 0)
        return;

    calculateBinary(n / 2, result);

    char bit = (n % 2) + '0';
    int len = strlen(result);
    result[len] = bit;
    result[len + 1] = '\0';
}

int main() {
    int n;
    char result[32] = "";

    printf("Enter an integer: ");
    scanf("%d", &n);

    if (n == 0) {
        strcpy(result, "0");
    } else {
        calculateBinary(n, result);
    }

    printf("Binary representation: %s\n", result);

    return 0;
}
```

Output

```
Enter an integer: 7
Binary representation: 111
PS C:\Users\kriti\OneDrive\Desk
nt-3-Kritigh62-bit\src>
```

Q29. Write a recursive function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```c
#include <stdio.h>
#include <string.h>

int isPalindrome(char str[], int start, int end) {

    if (start >= end)
        return 1;


    if (str[start] != str[end])
        return 0;

    // Recursive call
    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    char str[100];

    printf("Enter a string: ");
    scanf("%s", str);

    int len = strlen(str);

    if (isPalindrome(str, 0, len - 1))
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");

    return 0;
}
```

Output

```
 { .\q29 }
Enter a string: level
Palindrome
PS C:\Users\kriti\OneDrive\Desktop\C\assig
nt-3-Kritigh62-bit\src>
```

Q30. Write a recursive function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```c
#include <stdio.h>
int countOccurrences(char str[], char ch, int i);

int main() {
    char str[100], ch;

    printf("Enter a string: ");
    scanf("%[^\n]", str);

    printf("Enter a character: ");
    scanf(" %c", &ch);

    printf("Character '%c' occurs %d times.\n",
           ch, countOccurrences(str, ch, 0));

    return 0;
}

// Recursive function
int countOccurrences(char str[], char ch, int i) {
    if (str[i] == '\0')
        return 0;

    if (str[i] == ch)
        return 1 + countOccurrences(str, ch, i + 1);
    else
        return countOccurrences(str, ch, i + 1);
}
```

Output

```
Enter a string: clawclip
Enter a character: l
Character 'l' occurs 2 times.
PS C:\Users\kriti\OneDrive\Desktop\C\as
nt-3-Kritigh62-bit\src>
```